

Preprocessing Pipeline

E-Commerce Return Fraud Detection

Dataset: E-Commerce Return Fraud Detection

Records: 50,500 raw → 50,000 after cleaning

Features: 30 raw → 41 final modeling features

Target: return_fraud (0 = Genuine, 1 = Fraud)

Documented By: Ragavendra R

AI/ML Engineer

PREPROCESSING PIPELINE

Step 1: Duplicate Removal

Problem: 500 exact duplicate rows present in the raw dataset.

Observations:

- 500 rows were exact copies across all 30 columns.
- Duplicate order_id count matched duplicate row count exactly (500).
- Dataset reduced from 50,500 → 50,000 after removal.

Decision: Drop all exact duplicate rows using drop_duplicates(); reset index.

Reason:

- Identical rows provide zero additional information to the model.
- Retaining duplicates inflates sample weight for specific records and skews frequency statistics.

Alternatives Considered & Rejected:

- Keep duplicates → Rejected: Creates artificial over-representation of those records.
- Deduplicate by order_id only → Rejected: 500 IDs matched 500 full rows; result is identical but approach hides whether values differed.

Step 2: Categorical Inconsistency — gender and membership_type

Problem: Same categories represented in multiple formats.

Observations:

- gender had 6 variants: Male, M, male, MALE, Female, F, female.
- membership_type had 4 variants: Gold, GOLD, gold, Gld.
- Direct encoding without cleaning creates phantom categories treated as separate classes.

Decision: Strip whitespace → lowercase → map to canonical title-case value.

Reason:

- Strip prevents invisible leading/trailing spaces (e.g., ' Male') from producing silent NaN entries.

- Lowercase normalization collapses all case variants before the final mapping step.

Alternatives Considered & Rejected:

- `str.title()` directly → Rejected: Cannot handle abbreviations like 'M' or typos like 'Gld'.
- Label encode without normalization → Rejected: Would encode 'male' and 'Male' as separate numeric classes.

Step 3: Type Conversion — product_price (₹ Symbol)

Problem: product_price stored as object dtype due to embedded ₹ symbol.

Observations:

- Values like '₹1,299' cannot be read as numeric directly.
- `pd.to_numeric()` rejects non-numeric characters regardless of context.
- `errors='coerce'` needed to handle edge cases like '₹ N/A' without crashing.

Decision: Strip ₹ symbol as string operation first, then convert with `pd.to_numeric(errors='coerce')`.

Reason:

- String stripping before conversion handles all variants including '₹ ' (with space).
- `errors='coerce'` converts remaining unparseable values to NaN for imputation rather than raising exceptions.

Alternatives Considered & Rejected:

- `astype(float)` directly → Rejected: Throws `ValueError` on symbol-containing strings.
- Regex extraction → Rejected: Adds unnecessary complexity for a simple prefix strip.

Step 4: Type Conversion — discount_percent (% Symbol)

Problem: discount_percent stored as object dtype due to trailing % symbol.

Observations:

- Values like '15%' need the symbol removed before numeric conversion.
- Same root cause as product_price — storage artifact, not a data quality problem.

Decision: Strip % suffix as string, then convert with `pd.to_numeric(errors='coerce')`.

Reason:

- Consistent approach with product_price — strip symbol, coerce remainders to NaN.

Alternatives Considered & Rejected:

- `str.rstrip('%')` → Functionally equivalent; `str.replace` chosen for explicit intent.
- Divide by 100 during strip → Rejected: Type conversion and unit scaling are separate concerns.

Step 5: Invalid Values — product_price (Sentinel / Negative)

Problem: One row with product_price = 99,999,999 (data entry error); some negative prices.

Observations:

- 99,999,999 is a sentinel placeholder — no product is legitimately priced at ₹1 crore.
- Negative prices are mathematically impossible.
- Sentinel is captured later by the IQR outlier cap.

Decision: Set negative values to NaN; sentinel captured by IQR cap in Step 10.

Reason:

- Nulling invalid values preserves all other column data in those rows.

- Separating nullification from imputation keeps each step's purpose clear.

Alternatives Considered & Rejected:

- Drop rows entirely → Rejected: Other columns in those rows are valid.
- Cap to max valid price → Rejected: No reliable domain-based upper bound available.

Step 6: Invalid Values — customer_age (Out-of-Range)

Problem: 200 customer_age entries outside the valid 13–100 range.

Observations:

- Values below 13: legally implausible for an e-commerce account holder.
- Values above 100: statistically implausible; likely data entry errors.
- Without correction, these inflate age-based feature statistics.

Decision: Set out-of-range ages to NaN before imputation.

Reason:

- Nulling preserves the row's other columns.
- Imputation fills the gap with a defensible central value (median).

Alternatives Considered & Rejected:

- Clip to [13, 100] → Rejected: Keeps the extreme value rather than treating it as truly missing.
- Row deletion → Rejected: Discards 200 otherwise valid records.

Step 7: Invalid Values — return_rate > 1.0

Problem: 5,935 rows where return_rate exceeded 1.0 (mathematically impossible).

Observations:

- Root cause: previous_returns was larger than total_orders in these rows.
- A return rate cannot exceed 100% — you cannot return more than you ordered.
- These corrupt values directly feed the high_return_rate binary flag downstream.

Decision: Cap previous_returns at total_orders using np.minimum().

Recompute return_rate from scratch.

Reason:

- Fixing the source column ensures consistency across all return-related features.
- Recomputation guarantees return_rate is strictly in [0, 1] after this step.

Alternatives Considered & Rejected:

- Clip return_rate directly to [0, 1] → Rejected: Does not fix previous_returns, leaving a corrupted feature.
- IQR capping on return_rate → Rejected: Would incorrectly treat valid high-return customers as outliers.

Step 8: Date Format Standardization

Problem: All 4 date columns contained 5 mixed formats.

Observations:

- Formats present: YYYY-MM-DD, Mon-DD-YYYY, MM/DD/YYYY, DD/MM/YYYY, MM-DD-YYYY.
- A single fixed format string silently misparsed or errored on other convention rows.
- 24,854 rows had delivery_date earlier than order_date — nearly 50% of dataset.
- delivery_date corruption makes any interval feature derived from it unreliable.

Decision: Parse with pd.to_datetime(format='mixed', dayfirst=True, errors='coerce'). Drop delivery_date. Extract features from order_date only.

Reason:

- format='mixed' handles all 5 observed date conventions without manual detection.
- dayfirst=True resolves DD/MM vs MM/DD ambiguity in favor of regional format.
- errors='coerce' converts unparseable entries to NaT without stopping the pipeline.

Alternatives Considered & Rejected:

- Manual format detection per row → Rejected: Fragile and verbose for 5 formats.
- Keep corrupted delivery durations, clip negatives → Rejected: Clipping to 0 is misleading; negative duration has no physical meaning.

Step 9: Missing Value Imputation

Problem: 4 columns with missing values after type conversion and invalid value nulling.

Observations:

- customer_age: 7,529 missing (14.9%)
- membership_type: 6,750 missing (13.4%)
- product_price: 6,516 missing (12.9%)
- customer_rating: 7,575 missing (15.0%)
- product_price has extreme outliers (₹0 to ₹1 crore range) — mean would be heavily distorted.
- membership_type is categorical — no numeric center exists.

Decision: customer_age, product_price, customer_rating → Median imputation. membership_type → Mode imputation ('Gold').

Reason:

- Median is robust to extreme values in price-related fields — unaffected by the ₹1 crore outlier.
- Mode is the only sensible central tendency for a nominal categorical column.
- Result: 0 missing values across all columns after this step.

Alternatives Considered & Rejected:

- Mean imputation for numeric columns → Rejected: Mean distorted by outliers in product_price.
- KNN imputation → Rejected: Computationally expensive on 50,000 rows; negligible lift given near-symmetric distributions.
- Forward/backward fill → Rejected: Assumes temporal ordering which this dataset does not have.

Step 10: Outlier Capping

Problem: Extreme high-end values in transactional columns not yet handled by logical corrections.

Observations:

- total_orders: 51 outliers (0.1%)
- order_amount: 68 outliers (0.1%)
- return_rate outliers already eliminated in Step 7 — excluded here.
- Natural zeros (single-item orders, new accounts) are valid business data.

Decision: Apply IQR upper fence capping on total_orders, order_amount, product_price only. Formula: Upper cap = $Q3 + 1.5 \times IQR$.

Reason:

- Only the upper fence is applied — legitimate low values should not be clipped.
- return_rate deliberately excluded — already clean after Step 7.
- previous_returns indirectly bounded since it was capped at total_orders in Step 7.

Alternatives Considered & Rejected:

- Z-score capping (± 3 SD) → Rejected: More aggressive; removes valid high-volume customers.
- Log transformation → Rejected: Changes scale, making original units uninterpretable in deployed app.
- No outlier treatment → Rejected: Extreme values compress feature scale, reducing model discrimination.

Step 11: Feature Encoding

Problem: Model cannot process string categorical values directly.

Observations:

- gender: 2 categories, no inherent order → Binary.
- city_tier: 3 tiers with natural hierarchy (Tier 1 > 2 > 3) → Ordinal.
- membership_type: 3 tiers with hierarchy (Gold > Silver > Platinum) → Ordinal.
- product_category, payment_method, return_reason, device_type: Nominal, no ordering → One-hot.

- 4 nominal columns expand to 16 dummy columns with drop_first=True.
Decision: gender → Binary (Male=1, Female=0). city_tier, membership_type → Ordinal (1/2/3). Nominal columns → One-hot with drop_first=True.

Reason:

- Ordinal encoding preserves the real-world tier ordering that the model can exploit.
- drop_first=True eliminates perfect multicollinearity (dummy variable trap) in linear models.

Alternatives Considered & Rejected:

- One-hot for all categoricals → Rejected: Creates unnecessary dummies for tier columns where ordering carries real information.
- Label encoding for nominal columns → Rejected: Implies false ordinal relationships (e.g., Furniture > Electronics).

Step 12: Engineered Features

Problem: Raw columns do not directly expose behavioral patterns associated with return fraud.

Observations:

- account_age_days = 45 is harder to interpret than is_new_account = 1.
- support_tickets = 8 means very different things for a 10-order vs 150-order customer.
- Items returned within 2 days are rarely legitimate quality complaints.
- ip_risk_score 75 vs 65 has unclear significance without a threshold.

Decision: Create 6 behavioral features: account_age_years, is_new_account (<90 days), high_return_rate (>0.5), support_intensity (tickets/orders), high_ip_risk (score >70), fast_return (≤2 days).

Reason:

- support_intensity normalization makes metric volume-independent across all buyer volumes.
- 90-day new account window targets organized fraud ring account creation patterns.

- `fast_return` ≤ 2 days captures behavior inconsistent with legitimate quality-based complaints.
- `high_ip_risk` converts a noisy continuous signal into an operationally actionable binary flag.

Alternatives Considered & Rejected:

- Raw counts without normalization → Rejected: High-volume buyers appear riskier purely due to larger absolute ticket counts.
- Finer thresholds → Rejected: Would require holdout-set validation; chosen thresholds are operationally interpretable without overfitting.

Step 13: Data Leakage Removal

Problem: Several columns contain information unavailable at the time a return request is first received.

Observations:

- `fraud_investigation_status`: Output of post-return investigation — does not exist at prediction time.
- `refund_approved`: Decision made after fraud check — direct leakage.
- `final_fraud_decision`: The outcome itself — would make prediction trivial but useless in production.
- `refund_processed_date`: Only exists after a refund is processed — future information.
- These columns showed low correlation with `return_fraud` — but leakage is about TIMING, not correlation.

Decision: Drop: `fraud_investigation_status`, `refund_approved`, `final_fraud_decision`, `refund_processed_date`. Also drop: `order_id`, `customer_id` (identifiers), raw date columns.

Reason:

- A model trained on post-decision columns would score perfectly in training but produce zero value in production.
- All 41 final features are derivable from data present at the return request timestamp.

Alternatives Considered & Rejected:

- Keep low-correlation leakage columns → Rejected: Correlation is irrelevant; timing is the only valid leakage criterion.
- Use fraud_investigation_status with domain rules → Rejected: Status never available before the system makes its initial prediction.

Step 14: Train-Test Split and Feature Scaling

Problem: Model needs unseen data to evaluate real-world performance; scale differences affect gradient-based models.

Observations:

- Dataset: 50,000 rows, 70.3% fraud — stratification required to preserve class ratio.
- product_price ranges 0 to ~₹50,000; customer_age ranges 13–100 — different scales.
- Tree-based models (RF, XGBoost) do not require feature scaling.

Decision: 80/20 stratified split: 40,000 train / 10,000 test. StandardScaler fitted on training data only.

Reason:

- Stratification preserves the 70.3:29.7 class ratio in both sets.
- Scaler fitted only on training data — fitting on full dataset leaks test set statistics into training.

Alternatives Considered & Rejected:

- 70/30 split → Rejected: 80/20 with 50,000 rows gives a reliable 10,000-row test set.
- Min-Max scaling → Rejected: Sensitive to outliers; StandardScaler more robust given wide feature ranges.
- No scaling → Acceptable for tree models, applied selectively to Logistic Regression only.